Low Level Design

For

Multi-Agent Education System

Course Name: Agentic AI

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING

BY

Sr.no	Student Name	Enrolment Number
1	ABHIKET THAKUR	EN22CS301030
2	ADITYA PALIWAL	EN22CS301055
3	AARUSHI MITTAL	EN24CS3T10026
4	AKASH KUSHWAHA	EN22CS301083
5	AKSHAY DASHORE	EN22CS301096
6	AKSHITA RATHORE	EN22CS301100

Institution Name: Medi-caps University – Datagami Skill Based Course

Group: 04D1

Project No: AAI-04

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Nishant Shrivastava

Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

Jan - May 2026

1. Document Overview

1.1 Scope of the document

This document presents the **Low-Level Design (LLD)** of the Multi-Agent Education System and provides a detailed description of the internal architecture and implementation-level components of the system. It covers the design and interaction of various modules, including the user interface, application controller, agent definitions, task workflows, and multi-agent orchestration using CrewAI.

The document includes detailed specifications of system components such as class structures, function-level logic, data flow, and file handling mechanisms. It also describes the configuration and usage of the underlying language model integrated through Groq, along with the interaction between different agents responsible for research and content generation. Additionally, this document outlines API-level behaviour (if applicable), input-output formats, error handling strategies, and performance considerations required for the implementation of the system. It is intended to serve as a reference for developers to understand the system design in depth and to facilitate smooth implementation, testing, and future enhancements.

The document does not cover high-level architectural decisions, project planning, or user-level documentation, as those are addressed separately in the High-Level Design (HLD) and other project documents.

1.2 Intended Audience

The primary audience includes:

- Project Evaluators and Academic Reviewers
- Development Team Members
- Future Maintainers and Enhancers
- Research-Oriented Audience

1.3 System overview

The Multi-Agent Education System is an AI-driven application designed to generate structured and comprehensive study material for a user-defined topic. The system utilizes a collaborative multi-agent architecture, where each agent performs a specific role in the content generation pipeline. The implementation leverages the CrewAI framework to coordinate interactions between agents and manage task execution efficiently. The system begins by accepting a topic input from the user through a web-based interface developed using Streamlit.

This input is then processed by a sequence of agents. The Research Agent is responsible for collecting and organizing relevant information about the topic, including key concepts, explanations, and

examples. The Writer Agent subsequently transforms this information into a well-structured and easy-to-understand study guide.

The generated output is formatted into a readable structure and stored as a markdown file for future reference. The system is designed to provide a simple, efficient, and automated way for users to obtain high-quality educational content without manual research. Overall, the system demonstrates the application of multi-agent systems and large language models, integrated via Groq, to enhance learning experiences and automate knowledge generation.

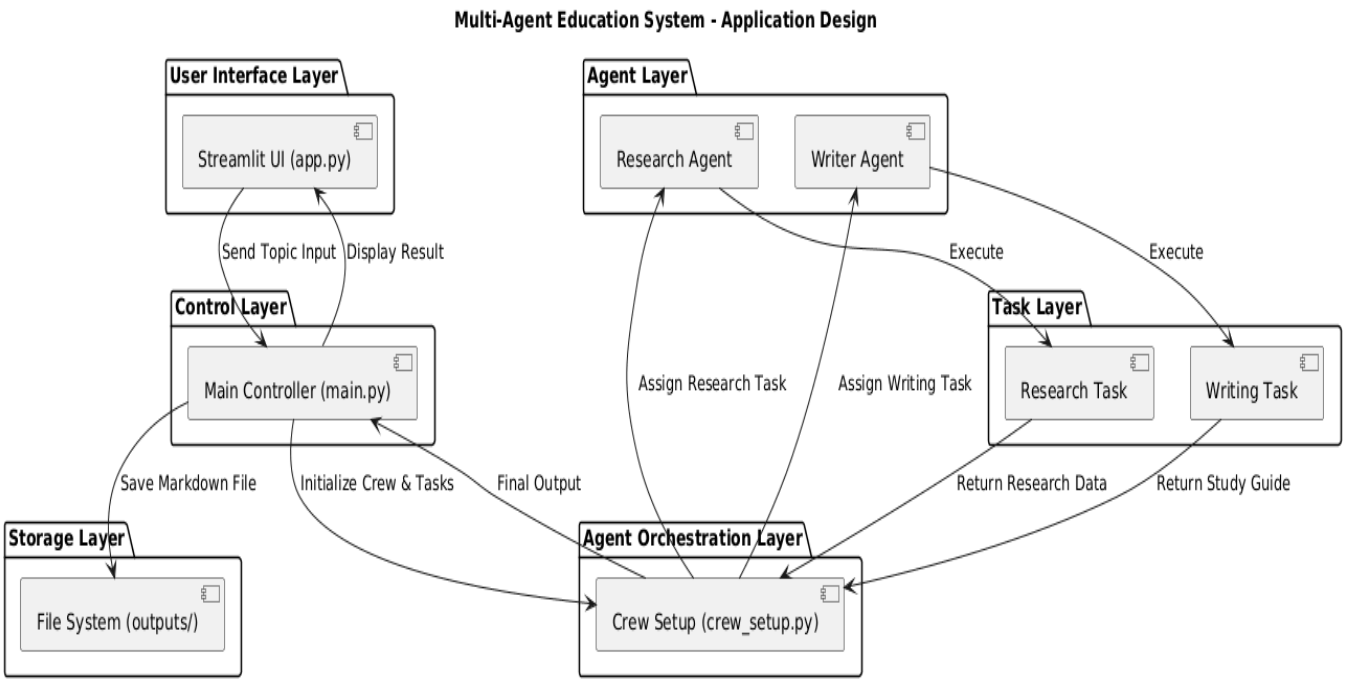
2. System Design

The System Design section provides a detailed description of the overall architecture and structural organization of the Multi-Agent Education System. It explains how different components of the system are designed, organized, and interact with each other to achieve the desired functionality.

This section focuses on the modular breakdown of the system, including the user interface, backend logic, agent definitions, and task orchestration implemented using CrewAI. It also outlines the data flow between components and highlights the design decisions taken to ensure scalability, maintainability, and efficient execution.

2.1 Application Design

The Application Design section describes the internal structure and behavior of the Multi-Agent Education System at the implementation level. It focuses on how the application components are developed and how they function together to deliver the required features.

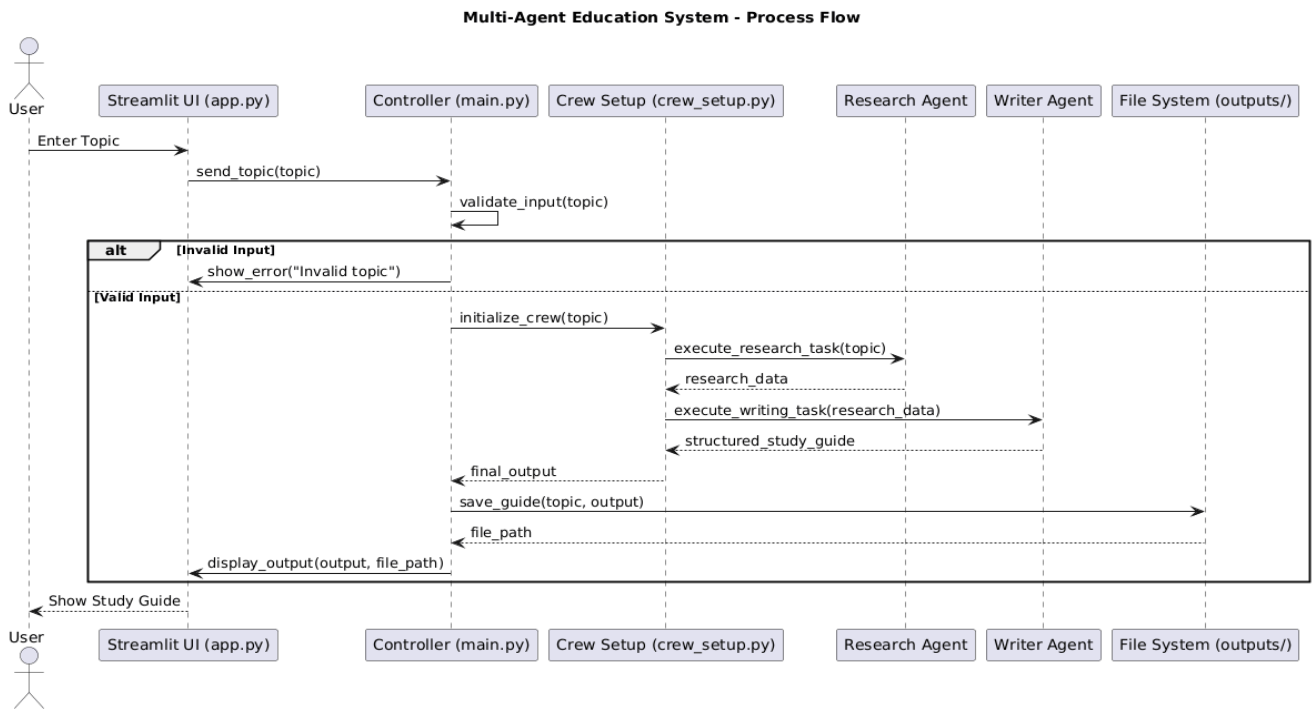


This section covers the design of individual modules, including the user interface built using Streamlit, the main control logic, and the integration of AI agents coordinated through CrewAI. It also explains the flow of data within the application, the interaction between functions and classes, and the handling of user input and generated output.

2.2 Process Flow

The Process Flow section describes the step-by-step sequence of operations performed within the Multi-Agent Education System. It explains how user input is processed, how tasks are executed by different agents, and how the final output is generated and delivered.

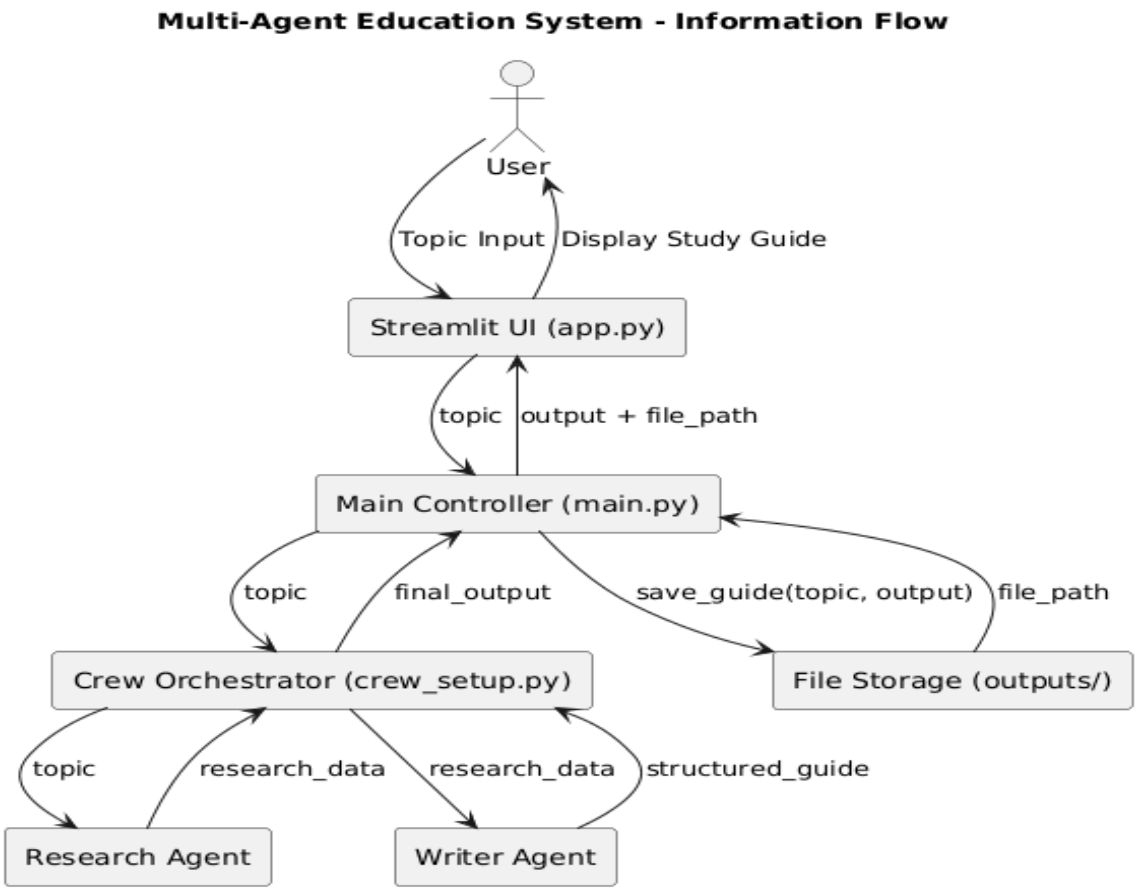
This section outlines the interaction between various components of the system, including the user interface, control logic, and agent orchestration implemented using CrewAI. It provides a clear understanding of how data flows through the system from input to output, ensuring transparency in execution and helping in debugging, testing, and optimization.



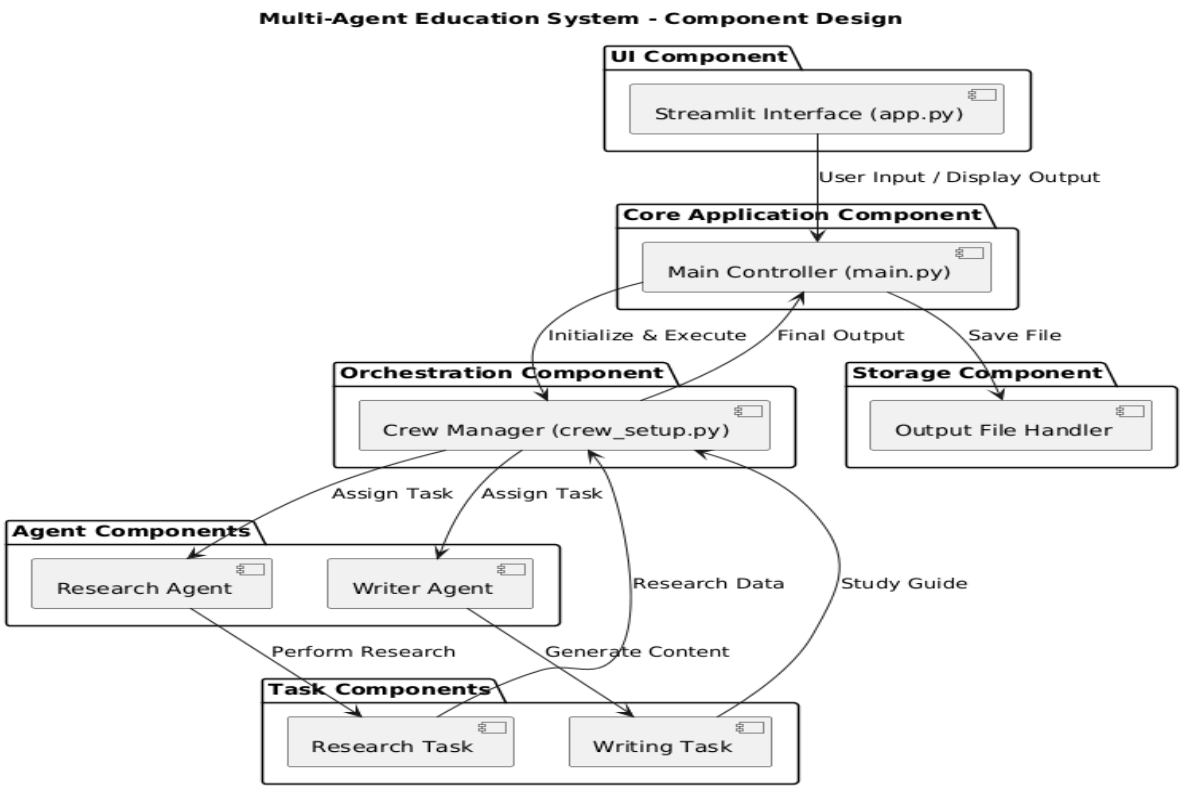
2.3 Information Flow

The Information Flow section describes how data moves through the Multi-Agent Education System from input to output. It explains how user-provided information is processed, transformed, and passed between different components of the system.

This section highlights the flow of data between the user interface, control logic, and the multi-agent framework implemented using CrewAI. It outlines how the input topic is converted into structured information by the Research Agent and further refined into a study guide by the Writer Agent.



2.4 Components Design



The Component Design section describes the individual building blocks of the Multi-Agent Education System and their responsibilities. It focuses on how the system is divided into reusable and independent components, each performing a specific function within the application.

This section outlines the interaction between core components such as the user interface, controller, agent modules, task definitions, and the orchestration layer implemented using CrewAI. The design ensures modularity, separation of concerns, and ease of maintenance, allowing components to be updated or extended independently without affecting the overall system.

2.5 Key Design Considerations

- Modularity and Separation of Concerns: The system is designed using a modular approach where different functionalities are divided into independent components such as the user interface, control logic, agents, and task definitions.
- Multi-Agent Architecture: A key design decision is the use of a multi-agent system implemented using CrewAI. Each agent is assigned a specific role, such as research or content generation, which improves clarity, reusability, and scalability. This approach also mimics real-world collaborative problem-solving.
- Sequential Task Execution: The system follows a sequential workflow where tasks are executed in a predefined order. The output of the Research Agent becomes the input for the Writer Agent. This ensures structured data processing and consistency in the generated output.
- Simplicity and Usability: The user interface is built using Streamlit to provide a simple and interactive experience. The design focuses on minimal user input and easy accessibility, allowing users to generate study material without technical complexity.
- Performance Optimization: The system uses lightweight agent configurations and optimized language model parameters (such as controlled token limits and temperature) to ensure faster response times. Efficient execution is important to maintain a smooth user experience.
- Scalability and Extensibility: The architecture is designed to be easily extendable. New agents (such as quiz generators or flashcard creators) can be added without modifying the existing system significantly. This ensures long-term scalability of the application.
- Data Handling and Storage: The system uses file-based storage (markdown files) to save generated content. Proper file naming conventions and directory management are implemented to ensure safe and organized storage.
- Error Handling and Reliability: Basic error handling mechanisms are incorporated to manage invalid inputs, API failures, and file handling errors. This improves system reliability and ensures graceful failure handling.
- Security Considerations: Sensitive information such as API keys is managed using environment variables. Input validation is performed to prevent invalid or unsafe data from entering the system.
- Maintainability: The code structure is organized into clearly defined modules and files, making it easier for developers to understand, modify, and enhance the system over time.

2.6 API Catalogue

2.6.1 Internal APIs (Functional Interfaces)

S.No	Name	Function	Description
1	Generate Study Guide API	generate_study_guide(topic: str)	Initiates the multi-agent workflow to generate a structured study guide for a given topic.
2	Save Guide API	save_guide(topic: str, content: str)	Saves the generated study guide into a markdown file
3	Safe Filename API	safe_filename(topic: str)	Converts the topic string into a safe file name by removing special characters.

2.6.2 External API Integration: LLM API (Groq)

Provider**:** Groq
Model Used**:** llama-3.1-8b-instant
Purpose**:** Used by agents to generate research content and structured study guides.

3. User Flow of the Application

- **Accessing the Application:** The user launches the application through the web interface built using Streamlit. The homepage displays a simple interface with an input field and a submission option.

- **Entering the Topic:** The user enters a topic (e.g., "Machine Learning", "Operating Systems") into the input field provided in the interface.

- **Submitting the Request:** After entering the topic, the user clicks the submit/generate button. This action sends the input topic to the backend controller for processing.

- **Input Validation:** The system validates the input to ensure that it is not empty or invalid.
  - If invalid → an error message is displayed to the user
  - If valid → the process continues

- **Triggering the Multi-Agent Workflow:** The validated topic is passed to the multi-agent orchestration layer implemented using CrewAI. The system initializes the agents and their respective tasks.

- **Research Phase (Research Agent):** The Research Agent processes the topic and performs the following:

  - Gathers relevant information
  - Identifies key concepts
  - Collects examples and explanations
  - Structures raw research data

  The output of this phase is a **research dataset**.

- **Content Generation Phase (Writer Agent):** The research data is passed to the Writer Agent, which:
  - Converts the research into a structured format
  - Organizes the content into sections
  - Simplifies explanations for better understanding
  - Prepares a readable study guide

  The output of this phase is the **final structured study guide**.

- **Output Processing and Storage:** The generated study guide is sent back to the main controller, where:
  - A safe filename is created
  - The content is saved as a markdown file in the outputs/ directory
  - The file path is generated for reference

- **Displaying Results:**The final output is sent back to the frontend. The Streamlit interface displays:
  - The generated study guide
  - Optionally, the file path or download option

- **User Consumption:** The user reads, downloads, or uses the generated study material for learning purposes.

4. References

- CrewAI Documentation – Used for implementing and understanding multi-agent architecture, agent roles, and task orchestration.
- Groq API Documentation – Referenced for integrating large language models and configuring parameters such as model selection, token limits, and response generation.
- Streamlit Documentation – Used for building the user interface and handling user interactions.
- Python Official Documentation – Used for core application development, file handling, and modular design.